

LAPORAN PRATIKUM 3
PEMOGRAMAN BEROENTAS OBJEK



Disusun Oleh:
Silva Nurifvan
2511533004

Dosen Pengampu :
Nurfiah, S.ST, M.Kom
Asisten Praktikum :
Siti Fadhilah Rahmi

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

BAB I PEMBAHASAN

1.1 Langkah Kerja Praktikum

1.1.1 Mengunci Entitas Transaksi

1) Pertama, buka aplikasi **Eclipse IDE for Java Developers**. Setelah itu, buat package baru dengan nama “**Pekan3**”, kemudian buat class baru bernama **Transaksi.java**. Selanjutnya, salin atau *copy-paste* kode dari class **Transaksi** yang telah dibuat pada praktikum pekan 2 ke dalam class tersebut.

```
1 package pekan2;
2
3 public class Transaksi {
4     String idTransaksi;
5     String jenis;
6     double nominal;
7
8     public Transaksi(String id,String jenis,double nominal)
9     {
10         this.idTransaksi = id;
11         this.jenis = jenis;
12         this.nominal = nominal;
13     }
14     public void cetakDetail() {
15         System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
16     }
17 }
```

2) Setelah itu ubah variabelnya menjadi private dan menambahkan Getter hanya untuk data yang perlu dibaca.

```
1 package pekan3;
2
3 import java.text.NumberFormat;
4 import java.util.Locale;
5
6 public class Transaksi {
7
8     private String idTransaksi;
9     private String jenis;
10    private double nominal;
11
12    // Constructor
13    public Transaksi(String id, String jenis, double nominal) {
14        this.idTransaksi = id;
15        this.jenis = jenis;
16        this.nominal = nominal;
17    }
18
19    // Getter (Read-Only)
20    public String getIdTransaksi() {return idTransaksi;}
21    public String getJenis() {return jenis;}
22    public double getNominal() {return nominal;}
23
24    // Menampilkan detail transaksi
25    public void cetakDetail() {
26        NumberFormat rupiah =
27            NumberFormat.getCurrencyInstance(new Locale("id", "ID"));
28        System.out.println(
29            "ID : " + idTransaksi +
30            " | Jenis : " + jenis +
31            " | Nominal : " + rupiah.format(nominal)
32        );
33    }
34 }
```

Class Transaksi pada tahap ini sudah menerapkan konsep enkapsulasi, yaitu atribut idTransaksi, jenis, dan nominal dibuat private sehingga tidak dapat diakses atau diubah secara langsung dari class lain.

Constructor digunakan untuk memberikan nilai pada atribut ketika objek Transaksi dibuat. Sementara itu, getter digunakan untuk mengambil atau membaca data transaksi tersebut.

Setter tidak digunakan karena objek Transaksi bersifat immutable, artinya data transaksi tidak dapat diubah setelah objek dibuat. Oleh karena itu, class ini hanya menyediakan getter sebagai akses untuk membaca data.

Method cetakDetail() masih memiliki fungsi yang sama seperti pada pekan sebelumnya, yaitu menampilkan informasi transaksi dengan nominal yang sudah diformat dalam bentuk Rupiah.

1.1.2 Mengunci Entitas Rekening dan Menambahkan PIN

1) Buat kembali class **Rekening** pada package **Pekan3** dengan menggunakan kode yang sama seperti pada praktikum sebelumnya. Setelah itu, lakukan perubahan dengan mengamankan atribut **saldo** dan menambahkan atribut **PIN** sebagai bagian dari penerapan konsep enkapsulasi.

```
1 package pekan3;
2
3 import java.text.NumberFormat;
4 import java.util.Locale;
5
6 public class Transaksi {
7
8     private String idTransaksi;
9     private String jenis;
10    private double nominal;
11
12    // Constructor
13    public Transaksi(String id, String jenis, double nominal) {
14        this.idTransaksi = id;
15        this.jenis = jenis;
16        this.nominal = nominal;
17    }
18
19    // Getter (Read-Only)
20    public String getIdTransaksi() {return idTransaksi;}
21    public String getJenis() {return jenis;}
22    public double getNominal() {return nominal;}
23
24    // Menampilkan detail transaksi
25    public void cetakDetail() {
26        NumberFormat rupiah =
27            NumberFormat.getCurrencyInstance(new Locale("id", "ID"));
28        System.out.println(
29            "ID : " + idTransaksi +
30            " | Jenis : " + jenis +
31            " | Nominal : " + rupiah.format(nominal)
32        );
33    }
34 }
```

```

package pekan3;

import java.text.NumberFormat;
import java.util.ArrayList;
import java.util.Locale;

public class Rekening {
    // Enkapsulasi dengan private
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin;

    private ArrayList<Transaksi> riwayatTransaksi;

    // Constructor
    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN
        if (pinValid(pinAwal)) {
            this.pin = pinAwal;
        } else {
            System.out.println(
                "Peringatan: PIN tidak valid. Menggunakan PIN default 583041"
            );
            this.pin = "583041";
        }

        this.riwayatTransaksi = new ArrayList<>();

        NumberFormat rupiah =
            NumberFormat.getCurrencyInstance(new Locale("id", "ID"));

        System.out.println(
            "Rekening atas " + namaPemilik +
            " berhasil dibuat dengan saldo " +
            rupiah.format(saldo)
        );
    }

    // Getter
    public String getNomorRekening() {

```

```

        return nomorRekening;
    }

    public String getNamaPemilik() {
        return namaPemilik;
    }

    // Validasi PIN
    public static boolean pinValid(String pin) {
        // PIN harus 6 digit
        if (pin == null || !pin.matches("\\d{6}")) {
            return false;
        }

        // Tidak boleh semua angka sama
        boolean semuaSama = true;

        for (int i = 1; i < pin.length(); i++) {
            if (pin.charAt(i) != pin.charAt(0)) {
                semuaSama = false;
                break;
            }
        }

        if (semuaSama) {
            return false;
        }

        // Tidak boleh berurutan naik atau turun
        boolean naik = true;
        boolean turun = true;

        for (int i = 1; i < pin.length(); i++) {
            int sebelumnya = pin.charAt(i - 1) - '0';
            int sekarang = pin.charAt(i) - '0';

            if (sekarang != sebelumnya + 1) {
                naik = false;
            }

            if (sekarang != sebelumnya - 1) {
                turun = false;
            }
        }

        if (naik || turun) {
            return false;
        }

        return true;
    }

    // Otentikasi PIN
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    // Setor tunai
    public void setorTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println(
                "Transaksi Gagal: Nominal setor harus lebih dari 0!"
            );
        } else {
            saldo += nominal;

            String idTrx =
                "TRX-S-" + System.currentTimeMillis();

            Transaksi transaksi =
                new Transaksi(idTrx, "Kredit", nominal);

            riwayatTransaksi.add(transaksi);

            NumberFormat rupiah =
                NumberFormat.getCurrencyInstance(
                    new Locale("id", "ID")
                );

            System.out.println(
                "Setor tunai " + rupiah.format(nominal) +
                " berhasil."
            );

            System.out.println(
                "Saldo saat ini : " + rupiah.format(saldo)
            );
        }
    }

```

```

    }

    // Tarik tunai
    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println(
                "Transaksi Gagal: Minimal nominal penarikan Rp10.000!"
            );
        } else if (nominal > saldo) {
            System.out.println(
                "Transaksi Gagal: Saldo tidak mencukupi."
            );
        } else if (saldo - nominal < 10000) {
            System.out.println(
                "Transaksi Gagal: Saldo minimal setelah penarikan Rp10.000!"
            );
        } else {
            saldo -= nominal;

            String idTrx =
                "TRX-T-" + System.currentTimeMillis();

            Transaksi transaksi =
                new Transaksi(idTrx, "Debit", nominal);

            riwayatTransaksi.add(transaksi);

            NumberFormat rupiah =
                NumberFormat.getCurrencyInstance(
                    new Locale("id", "ID")
                );

            System.out.println(
                "Tarik tunai " + rupiah.format(nominal) +
                " berhasil."
            );

            System.out.println(
                "Saldo saat ini : " + rupiah.format(saldo)
            );
        }
    }

    // Cek informasi rekening
    public void cekInformasi() {
        NumberFormat rupiah =
            NumberFormat.getCurrencyInstance(
                new Locale("id", "ID")
            );

        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir : " + rupiah.format(saldo));
        System.out.println("-----");
    }

    // Cetak riwayat transaksi
    public void cetakMutasi() {
        if (riwayatTransaksi.isEmpty()) {
            System.out.println(
                "Belum ada transaksi pada rekening ini."
            );
        } else {
            System.out.println("--- MUTASI REKENING ---");

            for (Transaksi trx : riwayatTransaksi) {
                trx.cetakDetail();
            }

            System.out.println("-----");
        }
    }
}

```

Pada class Rekening, dilakukan pengamanan data dengan menerapkan kembali konsep enkapsulasi. Atribut nomorRekening, namaPemilik, saldo, dan pin diubah menjadi private sehingga tidak dapat diakses secara langsung dari luar class.

Atribut pin ditambahkan sebagai data yang bersifat sensitif dan diberikan validasi agar PIN yang digunakan harus terdiri dari 6 digit.

1.2 Latihan/Tugas Pratikum

1.2.1 Deskripsi Tugas

Tugas: Mengamankan CLI dengan Otentikasi

Program Main.java diperbarui dengan menambahkan sistem keamanan berupa otentikasi PIN. Aturan keamanan yang diterapkan adalah sebagai berikut:

1. Menu 1 – Buka Rekening

Sebelum membuat atau menginstansiasi objek rekening baru, pengguna harus memasukkan PIN berupa angka yang terdiri dari 6 digit.

2. Keamanan Transaksi

Pada Menu 3 – Tarik Tunai dan Menu 6 – Cetak Mutasi, pengguna wajib memasukkan PIN terlebih dahulu sebelum sistem menjalankan proses tersebut.

Sistem akan memanggil method `akunAktif.otentikasi(pinYangDiinput)` untuk memeriksa kebenaran PIN. Jika hasilnya `true`, proses akan dilanjutkan.

Namun, jika hasilnya `false`, proses dibatalkan dan sistem menampilkan pesan:

"Akses Ditolak: PIN yang Anda masukkan salah!"

1.2.2 Perngerajaan Latihan/Tugas

1) Menu 1 (Buka Rekening): Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).

```
case 1:
    System.out.print("Masukkan No Rekening : ");
    String no = input.nextLine();

    System.out.print("Masukkan Nama Pemilik : ");
    String nama = input.nextLine();

    System.out.print("Masukkan Saldo Awal : ");
    double saldo = input.nextDouble();
    input.nextLine();

    String pin;
    while (true) {
        System.out.print(
            "Masukkan PIN Anda (6 digit): "
        );
        pin = input.nextLine();
        if (Rekening.pinValid(pin)) {
            break;
        }
        System.out.println(
            "PIN tidak valid!"
        );
        System.out.println(
            "PIN harus 6 digit, tidak boleh semua sama "
            + "atau berurutan."
        );
    }

    akunAktif =
        new Rekening(no, nama, saldo, pin);
    daftarRekening.add(akunAktif);
    break;
```

Pada **Menu 1**, ditambahkan proses input PIN sebelum objek rekening dibuat sesuai dengan aturan keamanan yang telah ditentukan.

Sebagai penerapan *challenge*, PIN yang dimasukkan juga diperiksa menggunakan method `pinValid()` yang terdapat pada class `Rekening`. Validasi ini memastikan PIN terdiri dari 6 digit serta tidak menggunakan angka yang sama secara berulang atau angka yang berurutan. Rekening hanya dapat dibuat apabila PIN yang dimasukkan sudah memenuhi semua aturan tersebut.

2) Perbarui Menu 3:

```
case 3:
    if (akunAktif == null) {
        System.out.println(
            "Error: Anda belum memiliki rekening!"
        );
    } else {
        System.out.print(
            "Masukkan PIN Anda: "
        );
        String pinInput = input.nextLine();
        if (akunAktif.otentikasi(pinInput)) {
            System.out.print(
                "Masukkan nominal yang akan ditarik: "
            );
            double tarik =
                input.nextDouble();
            input.nextLine();
            akunAktif.tarikTunai(tarik);
        } else {
            System.out.println(
                "Akses Ditolak: PIN yang Anda masukkan salah!"
            );
        }
    }
    break;
```

Pada **Menu 3**, sistem meminta pengguna memasukkan PIN terlebih dahulu sebelum method `tarikTunai()` dijalankan.

PIN yang dimasukkan kemudian diperiksa menggunakan `akunAktif.otentikasi(pinInput)`. Jika PIN benar, pengguna dapat memasukkan nominal dan proses penarikan tunai dilanjutkan. Namun, jika PIN salah, transaksi dibatalkan dan sistem menampilkan pesan "**Akses Ditolak: PIN yang Anda masukkan salah!**".

3) Perbarui Menu 6:

```
case 6:
    if (akunAktif == null) {
        System.out.println(
            "Error: Anda belum memiliki rekening!"
        );
    } else {
        System.out.print(
            "Masukkan PIN Anda: "
        );
        String pinInput = input.nextLine();
        if (akunAktif.otentikasi(pinInput)) {
            akunAktif.cetakMutasi();
        } else {
            System.out.println(
                "Akses Ditolak: PIN yang Anda masukkan salah!"
            );
        }
    }
    break;
```

Pada **case 6**, ditambahkan proses autentikasi PIN sebelum sistem menjalankan method cetakMutasi(). Sistem akan meminta pengguna memasukkan PIN, kemudian memeriksanya melalui akunAktif.otentikasi(pinInput). Jika PIN yang dimasukkan benar, riwayat transaksi dapat ditampilkan. Namun, jika PIN salah, akses akan ditolak dan riwayat transaksi tidak dapat ditampilkan.

1.2.3 Link Repository

https://github.com/SilvaNurifvan/PrakPBO_2026_2511533004/tree/24a11e640e001a82bf47e5ca0880db2e86cc1f66/pekan1/pekan3